TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI TP. HỒ CHÍ MINH (UTH)

KHOA CÔNG NGHỆ THÔNG TIN — BỘ MÔN CÔNG NGHỆ PHẦN MỀM

TOÀN VĂN GHI ÂM LỜI GIẢNG VIÊN DẶN DÒ & ĐÁNH DẤU TRỌNG TÂM

Học phần: Công nghệ Phần mềm (122005) | Bảo lưu nguyên văn 100% nội dung ghi âm chi tiết — Trực quan hóa các cảnh báo kỷ luật, quy chuẩn kỹ thuật và trọng tâm vấn đáp

---

Toàn bộ nội dung và thông tin bài giảng trong đoạn ghi âm được ghi lại chi tiết, đầy đủ và chuẩn xác theo từng phần như sau:

PHẦN I: HƯỚNG DẪN ĐẶC TẢ YÊU CẦU & THIẾT KẾ HỆ THỐNG PHÒNG KHÁM NHA KHOA

1. Quản lý phiên bản, kiểm soát thay đổi và Baseline (3 tuần đầu)

- QUY TẮC NHÓM **Làm việc trên nền tảng chung:** Các nhóm phải **làm việc trên cùng một nền tảng thống nhất**, tổ chức các buổi họp nhóm và họp review định kỳ.
- KIỂM TRA **Xác nhận bản thảo:** Cần kiểm tra và **xác nhận lại toàn bộ các bản thảo đặc tả yêu cầu** đã được lập bởi các thành viên trong nhóm.
- BẮT BUỘC **Quản lý phiên bản (Versioning) & Truy vết (Traceability):** Khi có bất kỳ thay đổi nào phát sinh, **bắt buộc phải tạo và phân chia các version** để dễ dàng truy vết lại lịch sử thay đổi một cách logic.
- Đây là các **yêu cầu cốt lõi bắt buộc** phải hoàn thiện trong giai đoạn 3 tuần này.
- Trong phần slide (Chương 2 về phân tích và thiết kế trên trang môn học), giảng viên đã cung cấp các nội dung baseline liên quan để sinh viên tải về tham khảo.

---

2. Phân tích chức năng thực tế: Quản lý lịch hẹn phòng khám nha khoa

- CỐT LÕI **Yêu cầu cốt lõi:** Lịch hẹn phải đảm bảo tính **đáng tin cậy**. Để giải quyết câu hỏi *"thế nào là đáng tin cậy"*, người phân tích phải mô hình hóa hệ thống thông qua việc xác định rõ các tác nhân (Actors) tham gia tương tác:
- **Lễ tân (Receptionist):** Đóng vai trò là người tiếp nhận, thiết lập (đặt) và thay đổi lịch hẹn.
- **Nha sĩ (Dentist):** Đưa ra lịch biểu làm việc khả thi của mình hoặc đăng ký các ca làm việc với phòng khám.
- **Bệnh nhân (Patient):** Xem hệ thống lịch (Calendar), tiến hành đăng ký lịch khám và theo dõi trạng thái lịch hẹn của mình. **Khi bệnh nhân đăng ký thành công, hệ thống phải hiển thị rõ thông tin lịch hẹn gồm ngày, giờ cụ thể và đích danh người khám.**

---

3. Quy trình chuyển đổi trạng thái (State Transition) của Lịch hẹn

Mỗi hành động của từng Actor sẽ tác động trực tiếp làm thay đổi trạng thái của lịch hẹn:

1. **Khởi tạo:** Khi lịch hẹn được đăng ký ban đầu (qua lễ tân hoặc bệnh nhân), lịch ở trạng thái **Tạo mới** (đã được tạo/confirm bước đầu).
2. **Xác nhận nội bộ:** Lễ tân có nhiệm vụ xác nhận (confirm) lại thông tin lịch hẹn với nha sĩ để chuyển trạng thái từ **Tạo mới** sang **Sẵn sàng**.
3. **Nha sĩ tiếp nhận:** Nha sĩ vào hệ thống xác nhận lịch đó và sẵn sàng cho ca khám, làm thay đổi trạng thái lịch hẹn.
4. **Thanh toán & Kết thúc:** Sau khi buổi khám diễn ra, người dùng xác nhận và hoàn tất thanh toán chi phí khám, trạng thái lịch hẹn chuyển sang **Hoàn thành / Kết thúc**.

** Một chức năng tưởng chừng đơn giản như "Đặt lịch" thực chất là một quy trình (process) liên tục có sự tương tác và làm thay đổi trạng thái bởi 3 Actor chính.*

---

4. Thiết kế hệ thống: Cấu trúc màn hình, Happy Case và Unhappy Case

MÔ HÌNH HÓA **Mô hình hóa hệ thống & cấu trúc màn hình:** Không thể chỉ có duy nhất một màn hình đặt lịch, mà phải **thiết kế đồng bộ nhiều màn hình phụ trợ**: màn hình tạo lịch, màn hình cho bác sĩ đăng ký ca trực, màn hình cho khách hàng theo dõi danh sách lịch hẹn... tạo thành một hệ thống quy tắc và quy trình hoàn chỉnh.

TRỌNG TÂM **Phân biệt Happy Case và Unhappy Case:**

- **Happy Case (Luồng lý tưởng):** Quy trình diễn ra trơn tru từ đầu đến cuối mà không gặp bất kỳ sự cố nào.
- **Unhappy Case (Luồng ngoại lệ / sự cố) - Bắt buộc phải xử lý:**
  - *Trường hợp khách hàng đặt lịch nhưng không đến (No-show):* Hệ thống phải xử lý trạng thái lịch và giải phóng tài nguyên ra sao?
  - *Trường hợp nha sĩ bận đột xuất hoặc gặp sự cố không thể tham gia ca khám:* Hệ thống có cho phép đổi sang nha sĩ khác không và luồng dữ liệu chuyển đổi như thế nào?
- **Người viết yêu cầu phải lường trước tất cả các kịch bản ngoại lệ này để đưa ra các ràng buộc cụ thể trong tài liệu đặc tả.**

- **Quy tắc hiển thị khung giờ (Slot):**
  - Những khung giờ còn trống (Available) thì cho phép người dùng click chọn và đăng ký.
  - Những khung giờ đã có người đặt thì **giao diện phải phản hồi rõ là đã được đăng ký, không cho bấm chọn tiếp**, giúp loại bỏ các lỗi thao tác của người dùng.

---

5. Cấu trúc chuyển hóa yêu cầu phần mềm (Requirement Hierarchy)

Quá trình phân tích yêu cầu phải đi theo **mô hình từ trên xuống (Top-down approach)**:

- **Business Requirement (Yêu cầu nghiệp vụ):** Mô tả bức tranh nghiệp vụ tổng quan mà hệ thống cần phục vụ (ví dụ: nghiệp vụ quản lý và đăng ký khám nha khoa).
- **Business Rules / Policy (Quy tắc nghiệp vụ):** Các quy định, chính sách bắt buộc tác động trực tiếp vào Business Requirement nhằm siết chặt nghiệp vụ và đảm bảo tính kín kẽ.
- **Scope Document (Phạm vi dự án):** Từ Business Requirement và Business Rule hình thành nên tổng quan phạm vi dự án, cần được làm rõ và ghi lại thật chi tiết.
- **User Requirements (Yêu cầu người dùng):** Được xây dựng dựa trên Scope Document và Business Rule. Mục này phải **chỉ rõ từng Actor khi vào hệ thống sẽ được phép làm gì, có quyền hạn nào và tương tác với hệ thống qua những bước cụ thể nào.**
- **Functional Requirements (Yêu cầu chức năng):** Chi tiết hóa toàn bộ các chức năng mà hệ thống phải xây dựng để người dùng thực hiện.
- **Non-functional Requirements / System Requirements (Yêu cầu phi chức năng / Hệ thống):** Những yếu tố kỹ thuật mà người dùng không nhìn thấy trực tiếp trên giao diện nhưng hệ thống bắt buộc phải đáp ứng:
  - Tốc độ truy cập và phản hồi của hệ thống.
  - **Khả năng chịu tải khi có nhiều người cùng truy cập tại một thời điểm (Concurrency).**
  - Khả năng sao lưu dữ liệu (Backup) và phục hồi hệ thống khi có sự cố (Recovery / Disaster Recovery).

---

6. Thiết kế giao diện (UI) và ứng dụng AI / Agent

- CÔNG CỤ **Công cụ thiết kế khuyến nghị:** Sử dụng **Figma** (`figma.com`) để thiết kế giao diện dựa trên danh sách các yêu cầu đã bóc tách.
- **Kết nối giữa Figma và Frontend:** Sau khi thiết kế xong trên Figma, có thể xuất giao diện sang mã nguồn HTML hoặc React.

- **Ứng dụng AI/Agent trong lập trình:** Có thể sử dụng AI Agent để đọc bản thiết kế/tài liệu và lập trình giao diện một cách nhanh chóng.

---

7. Bản chất môn học Công nghệ phần mềm và quy chuẩn tài liệu

> MỤC TIÊU MÔN **Mục tiêu môn học:** Môn Công nghệ phần mềm **không yêu cầu sinh viên phải "trâu bò" cắm đầu vào viết code**, mà đòi hỏi sự "trâu bò" trong việc **làm rõ yêu cầu (Requirements) và bóc tách cấu trúc hệ thống** một cách chặt chẽ.

- QUY CHUẨN GIT **Định dạng tài liệu nộp:**
  - Tài liệu đính kèm trong mã nguồn đẩy lên Git **phải được viết dưới dạng các file Markdown (`.md`)**, không dùng định dạng văn bản thông thường của các môn học trước.
  - Nội dung trong Markdown phải mô tả chi tiết, rõ ràng để AI / Agent có thể đọc hiểu và thực thi công việc được.
  - Sinh viên có thể tự viết tài liệu, sau đó nhờ AI review và sửa lại trước khi đưa cho agent triển khai.
- SẢN PHẨM CỐT LÕI **Sản phẩm cốt lõi:** Kết quả cuối cùng của chương này là bộ tài liệu đặc tả yêu cầu **SRS (Software Requirement Specification) hoàn chỉnh**, bao gồm đầy đủ các ràng buộc nghiệp vụ (Business Rules) để đảm bảo hệ thống vận hành kín kẽ và chính xác.

PHẦN II: NGUYÊN TẮC LÀM VIỆC NHÓM, ĐỊNH HƯỚNG VẤN ĐÁP & QUY CHẾ ĐIỂM LIỆT TỪ TUẦN SAU

1. Nguyên tắc phân công công việc và đánh giá sự hợp tác trong nhóm

> NGHIÊM CẤM **Bóc tách chi tiết công việc:** **Nghiêm cấm tuyệt đối việc chia việc theo kiểu "người A chỉ viết tài liệu (docs), người B làm slide/video, người C viết test...".** Toàn bộ khối lượng công việc bắt buộc phải được bóc tách thành từng phần, phân rã đến từng chi tiết nhỏ và giao trách nhiệm cụ thể đến từng thành viên.

- **Tính logic trong phân chia đầu việc:** Sự phân công cho từng thành viên phải có tính logic chặt chẽ. **Nếu phân công thiếu logic, giảng viên sẽ đánh giá nhóm không hiểu và không làm được việc, làm việc một cách hời hợt và thiếu sự phối hợp.** Nhóm phân công cẩu thả sẽ phải tự chịu trách nhiệm khi bị xử lý kỷ luật hoặc phạt vấn đáp.
- **Trách nhiệm tập thể:** Dù có phân công riêng, **giảng viên sẽ kiểm tra và vấn đáp cả nhóm**. Không chấp nhận tình trạng trong nhóm chỉ có một người làm hoặc cả nhóm bỏ mặc không ai làm.

---

2. Định hướng đánh giá và vấn đáp môn học: Tập trung vào Nghiệp vụ (Business)

> ĐỊNH HƯỚNG THI **Trọng tâm là giải quyết bài toán nghiệp vụ:** Giảng viên sẽ **xoáy sâu vào bản chất nghiệp vụ (business)**. Cụ thể, giảng viên sẽ đặt ra một vấn đề/sự cố thực tế (problem) và yêu cầu sinh viên phải trình bày rõ phương án giải quyết bài toán đó ra sao.

- **Không kiểm tra kỹ thuật lập trình cơ bản:** Giảng viên sẽ không đi sâu vào các vấn đề kỹ thuật thuần túy (technical) như: đoạn code này viết như thế nào, chức năng này gọi ra sao, hay bắt xóa code đi để gõ lại. Giảng viên coi các bài kiểm tra gõ code này là cấp độ cơ bản (*"đồ trẻ con"*).
- **Quan điểm về việc viết code và AI:** Việc sinh code hiện nay đã có các công cụ AI hỗ trợ thực hiện, do đó giảng viên không quá bận tâm đến việc gõ code thủ công. **Giá trị cốt lõi của người học nằm ở tư duy phân tích và thiết kế nghiệp vụ.**

---

3. Yêu cầu nghiệp vụ (Business Requirements) và xử lý kịch bản ngoại lệ (Unhappy Cases)

- **Bản chất của Business Requirement:** Là kết quả của quá trình vận hành nghiệp vụ và là lý do cần phải cải tiến, thay đổi. Khi nghiệp vụ thay đổi, toàn bộ các thành phần liên quan trong hệ thống cũng sẽ phải thay đổi theo.
- **Giải quyết xung đột nghiệp vụ (Conflict Handling):** Nghiệp vụ thực tế phát sinh thay đổi liên tục. Hệ thống phải đưa ra các giải pháp để giảm thiểu tối đa các xung đột phát sinh trong quá trình vận hành.

BÀI TOÁN VÍ DỤ **Bài toán ví dụ về đăng ký lịch hẹn / giữ chỗ:**

- *Tình huống đặt ra:* Trong cùng một thời điểm, có nhiều người (ông A, ông B, ông C) cùng truy cập vào một khung giờ (slot) đang trống và cùng gửi yêu cầu đăng ký. Nếu không có giải pháp xử lý, hệ thống sẽ bị lỗi trùng lịch. Hệ thống phải xác định rõ tiêu chí giải quyết cho ai và từ chối ai.
- *Giải pháp thực tế (tương tự mô hình đặt vé tàu, vé máy bay):* **Khi người dùng click chọn slot, hệ thống phải cho phép giữ chỗ tạm thời (lock/hold) trong một khoảng thời gian nhất định (khoảng 5 đến 10 phút) kể cả khi họ chưa hoàn tất việc thanh toán hay nhập form.** Nếu sau 5–10 phút mà người đó không hoàn tất đơn, slot này sẽ tự động được mở khóa (unlock) để giải phóng cho người khác đăng ký. Điều này khác hoàn toàn với việc người dùng đang xem dở thì bị người khác chen ngang đăng ký mất.

TRỌNG TÂM VẤN ĐÁP **Hình thức vấn đáp trọng tâm:** Khi vấn đáp, giảng viên sẽ **chủ yếu xoáy sâu vào các Unhappy Cases**. Thành viên nào không tham gia trực tiếp vào quy trình làm việc nhóm thì chắc chắn sẽ không thể đưa ra được phương án xử lý.
Tư duy xử lý Unhappy Cases: Chỉ khi thực sự làm việc nhóm sâu sát thì các thành viên mới phát hiện ra vô số trường hợp ngoại lệ (unhappy cases) và tình huống lỗi phát sinh cần phải xử lý. Tư duy kỹ sư nằm ở khả năng giải quyết các kịch bản bất thường này, chứ không phải chỉ dừng lại ở việc tạo ra một màn hình gồm các nút chức năng A, B, C thông thường (những thứ AI có thể tự sinh ra được).

- **Tiêu chí của yêu cầu thay đổi:** Mọi yêu cầu thay đổi nghiệp vụ đưa ra đều phải đo lường được, thống nhất về các trạng thái chuyển đổi, xác định rõ ràng mục tiêu và mốc thời gian thực hiện.

4. Quy chế kiểm tra tiến độ và kỷ luật từ tuần sau

QUY CHẾ KỶ LUẬT ĐIỂM LIỆT — BẮT ĐẦU TỪ TUẦN SAU

- **Quy định đối với tuần hiện tại:** Tuần này giảng viên mang tính chất nhắc nhở, có thể châm chước bỏ qua.
- **Quy chế kiểm tra nghiêm ngặt từ tuần sau:** Bắt đầu từ tuần sau, giảng viên sẽ kiểm tra trực tiếp từng nhóm.
- **Hình thức kiểm tra:** Gọi ngẫu nhiên (random) vào đầu giờ, giữa giờ; hoặc trong quá trình giảng bài, nếu phát hiện nhóm nào, sinh viên nào mất tập trung thì sẽ gọi lên vấn đáp ngay lập tức. Nếu sinh viên được gọi không trả lời được, giảng viên sẽ tiến hành kiểm tra toàn bộ nhóm đó.
- **Mức phạt:** Từ tuần sau, nếu kiểm tra mà nhóm không có tiến độ/kết quả theo yêu cầu thì sẽ **BỊ ĐÁNH ĐIỂM LIỆT CẢ NHÓM (RỚT MÔN / HỦY KẾT QUẢ)**.
- **Quy định "không xé nháp":** Giảng viên tuyên bố tuyệt đối **không có lần đầu, không có làm nháp hay thông cảm làm lại**, vì sinh viên đã có trọn vẹn 4 tuần chuẩn bị.
- Từ tuần thứ 5, thứ 6 trở đi, hoạt động kiểm tra tiến độ sẽ diễn ra liên tục qua từng tuần để kiểm soát gắt gao kết quả thực hiện của từng nhóm.

** Thực tế triển khai so với tài liệu lý thuyết: Slide bài giảng chỉ tóm tắt 4–5 dòng ngắn gọn về baseline, nhưng khi bắt tay vào thực hiện thực tế thì đòi hỏi nỗ lực rất lớn của toàn nhóm.*

PHẦN III: QUY TRÌNH PHÁT TRIỂN YÊU CẦU, CƠ CẤU NHÂN SỰ & THIẾT KẾ UI FIGMA

1. Quy trình phát triển yêu cầu phần mềm và các vai trò tham gia

- **Mức độ yêu cầu nghiệp vụ (Business Requirements):** Đội ngũ quản lý (Manager) và nhân viên kinh doanh/Marketing trực tiếp làm việc với khách hàng để thu thập và làm rõ yêu cầu nghiệp vụ. Ở giai đoạn này, các yêu cầu mới chỉ dừng lại ở mức độ tổng quan (High-level) chứ chưa đi vào chi tiết kỹ thuật.
- **Mức độ yêu cầu người dùng (User Requirements):** Chuyên viên phân tích nghiệp vụ (Business Analyst - BA) và Quản lý dự án (Project Manager - PM) tiếp nhận và phân tích sâu hơn. Nhiệm vụ chính là xác định rõ: **Ai tham gia vào hệ thống, họ thực hiện những công việc gì, làm như thế nào và tính logic chặt chẽ của các luồng nghiệp vụ ra sao.**
- **Mức độ yêu cầu chức năng (Functional Requirements):** Quá trình chuyển đổi từ yêu cầu người dùng sang yêu cầu chức năng trải qua các bước: Làm sáng tỏ toàn bộ khúc mắc (*"Clear" vấn đề*) và làm mịn, chuẩn hóa chi tiết nhất (*"Clean" vấn đề*) để sẵn sàng cho khâu lập trình (Develop). Đội ngũ lập trình viên (Developer) và kiểm thử viên (Tester) bắt đầu tham gia sâu từ bước triển khai này.
- **CHU TRÌNH SRS** Cốt lõi của tài liệu SRS được đúc kết qua chu trình: **Clear** (làm rõ hết các vấn đề) → **Clean** (phân tích chi tiết nhất có thể) → **Develop** (tiến hành phát triển).

2. Cơ cấu nhân sự dự án phần mềm và yêu cầu phân công công việc

- **Cơ cấu nhóm dự án tiêu chuẩn (Project Team):** Một nhóm phát triển cơ bản gói gọn gồm: PM, BA, Developer và Tester. Trong các mô hình nhóm quy mô nhỏ: BA có thể kiêm nhiệm luôn vai trò Tester; Developer kiêm nhiệm luôn việc viết Unit Test.
- **Vị trí Kiến trúc sư ứng dụng / Kiến trúc sư giải pháp kỹ thuật (Application Architect / Technical Architect - thường tương đương Giám đốc công nghệ CTO):** Đảm nhiệm việc định nghĩa kiến trúc hệ thống phần mềm, thường tham gia vào một giai đoạn nhất định hoặc quản lý kiến trúc xuyên suốt nhiều dự án cùng lúc.
- Bộ phận làm việc thường trực trong dự án gồm PM và BA (làm việc trực tiếp với khách hàng; BA là cầu nối giữa khách hàng và nhóm kỹ thuật); Developer và Tester tập trung liên tục vào công đoạn kỹ thuật.

CẢNH BÁO PM **Quy định phân công công việc trong nhóm sinh viên:**

- Giảng viên **nghiêm cấm tình trạng sinh viên nhận vai trò PM chỉ để "chỉ tay năm ngón", giao tiếp và phân việc**. Nếu giữ vị trí đó thì phải thể hiện được trình độ chuyên môn nghiệp vụ rất sâu, và khi vấn đáp giảng viên sẽ hỏi xoáy thẳng vào nghiệp vụ kinh doanh và quy trình.
- **Tất cả các thành viên trong nhóm bắt buộc đều phải bắt tay vào làm việc cụ thể.**
- Thời lượng môn học là 8 tuần nhưng thực tế thời gian để làm sản phẩm chỉ có khoảng 4 đến 5 tuần, do đó công việc phải được phân bổ đều và chi tiết để tránh việc chỉ một người gánh toàn bộ nhóm.

TIÊU CHÍ CHẤM **Tiêu chí chấm điểm đồ án:**

- Giảng viên sẽ **đánh giá tổng quan sản phẩm cuối cùng xem có hoàn thiện và hoạt động được hay không trước khi xét đến đóng góp của từng cá nhân**.
- **Nếu sản phẩm không hoàn thành hoặc không có gì để chấm thì không xét đến các vị trí quản lý hay đóng góp chi tiết.**

3. Yêu cầu chức năng và Thiết kế giao diện (UI)

- **Định nghĩa Yêu cầu chức năng (Functional Requirements):** Functional Requirement là sự kết hợp chặt chẽ giữa các quy tắc nghiệp vụ (Business Rules / Constraints) và phản hồi tương ứng của hệ thống (System Response). Tài liệu phải mô tả rành mạch: Chức năng đó làm nhiệm vụ gì và kết quả đạt được là gì.

QUY CHUẨN UI **Quy chuẩn thiết kế UI trên Figma:**

- **Toàn bộ giao diện bắt buộc phải được thiết kế trên Figma.**
- Quy trình chuẩn là phải **phân tích và vẽ thiết kế giao diện trên Figma trước khi bắt đầu lập trình**. Giảng viên sẽ kiểm tra tiến độ thiết kế trước mã nguồn, **không chấp nhận việc viết code xong xuôi rồi mới quay lại vẽ giao diện hay viết tài liệu bổ sung**.
- Các công việc gồm phân tích, thiết kế, lập trình và kiểm thử phải được triển khai song song liên tục.

4. Các yêu cầu phi chức năng (Non-functional Requirements)

- **Hiệu năng (Performance):** Người dùng không quan tâm hệ thống dùng thuật toán nào bên dưới, họ chỉ đánh giá hệ thống phản hồi nhanh đến mức nào và khả năng chịu tải được bao nhiêu người dùng đồng thời.
- **Tính sẵn sàng (Availability):**
  - Đánh giá mức độ hoạt động 24/7 của hệ thống và thời gian ngừng hoạt động (downtime) khi bảo trì.
  - *So sánh kiến trúc:* Trong các hệ thống TMĐT lớn áp dụng kiến trúc Microservices, khi cần bảo trì thì hệ thống chỉ tạm dừng một vài dịch vụ cụ thể mà không làm gián đoạn toàn bộ hệ thống; ngược lại, nếu dùng kiến trúc nguyên khối (Monolithic), một dịch vụ gặp sự cố hoặc tắt đi bảo trì sẽ khiến toàn bộ hệ thống sụp đổ.
- **Bảo mật (Security):** Phải thiết lập các Business Rules cụ thể để giải quyết rành mạch hai câu hỏi: **Ai được phép truy cập vào dữ liệu nào? Ai được quyền thực thi chức năng nào?** Tất cả phải được mô tả kỹ lưỡng thông qua các kịch bản Use Case.
- **Mẫu tài liệu (Template):** Giảng viên đã tải sẵn biểu mẫu đặc tả lên hệ thống môn học ngay từ tuần đầu tiên, sinh viên phải tự tải về nghiên cứu để làm theo thay vì hỏi lại.

5. Ràng buộc nghiệp vụ (Business Rules), Khoảng đệm (Buffer Time) và Giữ chỗ (Locking)

- **Quy tắc xếp lịch phòng khám nha khoa:**
  - Chính sách (Policy) quy định rõ: Một nha sĩ không thể có các lịch hẹn bị trùng nhau, trong một khung giờ (slot) không thể tiếp nhận đồng thời hai bệnh nhân.
  - **Bắt buộc phải có khoảng đệm thời gian trống (Buffer Time) giữa các slot (khoảng 15 đến 30 phút)** để dự phòng trường hợp ca khám trước kéo dài vượt giờ, tránh làm ảnh hưởng hoặc đè lên ca khám tiếp theo.
- **Hành vi xử lý của hệ thống (System Behaviors):**
  - Hệ thống phải có cơ chế từ chối các yêu cầu đặt lịch không hợp lệ hoặc bị xung đột.
  - Phải thực hiện kiểm thử biên (Boundary test) tại các điểm giới hạn trên và dưới của từng slot giờ.
- **Giao diện trao đổi và Cơ chế giữ chỗ tạm thời (Holding Slot):**
  - Tương tự cơ chế của các hệ thống bán vé máy bay hoặc vé xem phim: Khi người dùng click chọn một slot giờ, hệ thống phải kích hoạt trạng thái giữ chỗ tạm thời trong một khoảng thời gian nhất định (ngay cả khi người dùng chưa thanh toán) để ngăn chặn người khác đặt đè lên.
  - Kịch bản xử lý này và quy trình kiểm thử phải được thể hiện chi tiết trên bản vẽ mẫu (prototype) của Figma.

6. Ý thức học tập của sinh viên và ứng dụng công cụ

- Sinh viên năm 2, năm 3 (khoảng 19 - 20 tuổi) đã ở độ tuổi trưởng thành, cần chủ động nghiên cứu và thực hành nhiều.
- Các công cụ phần mềm hiện nay (như Figma) đều hỗ trợ thao tác kéo thả, tạo liên kết prototype và đã tích hợp nhiều tính năng hỗ trợ từ AI.
- **Giảng viên không chấp nhận các lý do như "em không biết" hay than vãn khó khăn**; sinh viên không làm được chỉ vì chưa thực sự dành thời gian để nghiên cứu và thực hiện.

7. Cấu trúc môn học, Thang điểm và Đánh giá kết quả

THANG ĐIỂM **Cơ cấu điểm số & Thang điểm:**

- Điểm thi kết thúc học phần chỉ chiếm **30%** tổng điểm.
- **Điểm quá trình chiếm tỷ trọng lớn (70%) và phụ thuộc hoàn toàn vào kết quả thực hiện Đồ án (Project).**
- **Kể cả khi thi lý thuyết đạt 10 điểm nhưng điểm quá trình bị đánh giá thấp (ví dụ 4.5, 4.8 hoặc 5.5 do không tham gia làm đồ án nghiêm túc) thì vẫn bị trượt môn.** Nếu nhóm làm việc đàng hoàng, nghiêm túc thì giảng viên sẽ đánh giá đúng năng lực chứ không đánh điểm thấp.

- **Tài liệu và Ngôn ngữ:** Slide bài giảng sử dụng hoàn toàn bằng tiếng Anh để bám sát đề cương chi tiết và các giáo trình tham khảo chuẩn đã được nhà trường công bố.
- **Tiến độ giảng dạy:** Theo đề cương (Syllabus), chương này được bố trí dạy trong 3 buổi. Tiến độ giữa các lớp được giảng viên cân đối đồng bộ về số lượng slide và khối lượng bài tập để đảm bảo tính công bằng giữa các nhóm.

---

8. Xử lý kịch bản ngoại lệ (Unhappy Cases) thông qua Hệ thống thông báo (Notification)

- Các ràng buộc hệ thống cần có sự kết hợp của cơ chế gửi thông báo (Notification / Alert).
- **Tình huống thực tế:** Khi bệnh nhân đặt lịch thành công, hệ thống cần gửi thông báo nhắc nhở trước ngày/giờ khám. Nếu bệnh nhân quên và không đến khám, lịch hẹn sẽ rơi vào kịch bản ngoại lệ (Unhappy Case) khiến ca khám bị thất bại (Failed) và lịch bị hủy bỏ.
- **Việc thiết lập cơ chế thông báo tự động (tương tự các ứng dụng di động như MoMo hay hệ thống lịch trên điện thoại) là giải pháp nghiệp vụ then chốt để hạn chế tối đa các Unhappy Cases xảy ra trong quá trình vận hành hệ thống.**

PHẦN IV: QUẢN LÝ MÃ NGUỒN, TÀI LIỆU VÀ BÀI TOÁN SINH TỒN NGHỀ NGHIỆP

1. Kiến trúc phần mềm và quản lý mã nguồn (Backend & Frontend)

- **Thiết kế kiến trúc hệ thống:** Trong chương Software Design, giảng viên sẽ hướng dẫn các mô hình kiến trúc phần mềm phổ biến. Sinh viên có nhiệm vụ nghiên cứu sâu về các kiến trúc này và **tạo một file `README.md` riêng cho phần Backend**.
- **Đặc tả Frontend:** Tương tự Backend, sinh viên cần xác định rõ Frontend sử dụng kiến trúc gì, công nghệ nào và **mô tả cụ thể vào một file `README.md` của Frontend**. Thao tác này chỉ mất khoảng 5 phút để hoàn thành, trọng tâm là phải định nghĩa (define) thật chi tiết.
- **Giao tiếp giữa Backend và Frontend:** Sinh viên phải làm rõ giao thức kết nối (giao thức HTTP) và viết các script mẫu để hệ thống chạy và thực thi từng bước một.
- **Lý do phân chia công việc trong nhóm:** Mỗi sinh viên đều bị giới hạn về lượng token khi sử dụng các công cụ AI (token không phải vô hạn), do đó một người không thể tự chạy hết mọi tác vụ mà **bắt buộc cả nhóm phải chia nhỏ đầu việc để cùng làm**.

---

2. Bóc tách yêu cầu và quản lý tài liệu

- **Đóng góp của sinh viên:** Phần đóng góp lớn nhất của sinh viên là phải viết các yêu cầu (Requirements) thật chi tiết.
- **Đồng bộ hóa tài liệu:** Từ các yêu cầu chi tiết đã phân tích, sinh viên phải cập nhật ngược lại vào hệ thống tài liệu trên nền tảng Confluence.
- CẤU TRÚC DOCS **Cấu trúc thư mục Docs:** **Trong thư mục `Docs/` của dự án phải chứa nhiều file `README.md` bóc tách từng phần. Mỗi khi có sự thay đổi, bắt buộc phải cập nhật và đánh số phiên bản (version) liên tục chứ không phải chỉ viết một lần là xong.**

- **Quy trình kết hợp với AI:** Sinh viên nên tự tay viết bản thảo yêu cầu trước, sau đó mới dùng AI để tinh chỉnh (refine) lại câu chữ theo đúng ý đồ của mình. **Tuyệt đối không phó mặc hoàn toàn cho AI viết từ đầu đến cuối, vì nếu không trực tiếp làm thì đến lúc vấn đáp sẽ không hiểu gì để trả lời.**

3. Quy chuẩn vấn đáp môn học

QUY CHUẨN THI **Trọng tâm kiểm tra vấn đáp:**

- Giảng viên khẳng định **chỉ tập trung vấn đáp sâu về mặt nghiệp vụ (Business) của hệ thống**.
- **Không kiểm tra kỹ thuật lập trình:** Giảng viên sẽ không hỏi về công nghệ hay cú pháp code, trừ trường hợp sinh viên có nguyện vọng muốn được vấn đáp về code.
- **Yêu cầu sinh viên:** Sinh viên cần nghiêm túc ngồi lại để phân rã (breakdown) cấu trúc tài liệu và xác định rõ cách viết chi tiết cho từng phần.

4. Năng lực cạnh tranh và bài toán sinh tồn tại thành phố

TÂM SỰ NGHỀ NGHIỆP **Tâm sự thực tế của Giảng viên về Năng lực cạnh tranh và Bài toán sinh tồn:**

- **Thái độ học tập:** Giảng viên phê bình thái độ than vãn *"khó quá không học được"* hay *"không làm được"*. Vấn đề then chốt nằm ở việc sinh viên đã dành bao nhiêu thời gian cho việc học và có thực sự đam mê với ngành hay không.
- **Yếu tố đam mê:** Nếu không có đam mê thì không thể theo đuổi ngành công nghệ phần mềm lâu dài, kể cả sau khi tốt nghiệp ra trường.
- **Con đường phát triển:** Ngoại trừ những trường hợp có quan hệ gia đình (con ông cháu cha - COCC) trong các tập đoàn lớn để có sẵn việc làm sau khi nhận bằng, các sinh viên đi lên từ năng lực tự thân (*"từ chân phương, chân đất"*) muốn bám trụ lại thành phố **chỉ có một con đường duy nhất là chứng minh bằng thực lực và năng lực thực tế**.

Đối thoại thực tế về chi phí sinh hoạt (Giảng viên trao đổi cùng sinh viên):

Giảng viên đặt câu hỏi: *Sau khi ra trường trong 5 năm đầu, nếu bố mẹ không chu cấp tiền nữa thì sinh viên sẽ sống như thế nào?*

- Tiền thuê phòng trọ: Khoảng 2 triệu đồng/tháng.
- Tiền ăn uống: Khoảng 2 triệu đồng/tháng.
- Tiền sinh hoạt, chi tiêu khác: Khoảng 1 triệu đồng/tháng.
- → **Tổng chi phí tối thiểu: 5 triệu đồng/tháng**.
- **Mức lương kỳ vọng của sinh viên:** Khoảng **6 triệu đồng/tháng**.

Phân tích rủi ro tài chính của giảng viên:

- Mức lương 6 triệu đồng là mức rất bấp bênh; nếu ốm đau, bệnh tật thì không có tiền mua thuốc.
- Nếu gặp rủi ro phát sinh như xe máy hỏng giữa đường thì lại rơi vào cảnh phải xin tiền gia đình dưới quê gửi lên để sửa xe.
- Để có thể tồn tại được với mức thu nhập 6 triệu đồng tại thành phố: Phải ở ghép từ 3 đến 4 người trong một phòng để chia nhỏ tiền phòng, phải chi tiêu cực kỳ tiết kiệm, và phải rèn luyện thể dục thể thao để giữ gìn sức khỏe, tránh ốm yếu quặt quẹo.
- Với mức chi phí 5 triệu và lương 6 triệu, trong 2 năm đầu tiên xác định phải sống độc thân vì chưa có điều kiện để hẹn hò hay lo cho người khác.

👉 **Lời khuyên hành động: Cuộc sống thực tế rất khắc nghiệt, sinh viên (đặc biệt là các bạn nam) bắt buộc phải nỗ lực cày ải gấp đôi, gấp ba ngay từ thời điểm hiện tại. Nếu chỉ biết ăn uống xong rồi ngồi chơi game qua ngày thì sau khi học xong chắc chắn sẽ phải bỏ về quê.**

5. Hướng dẫn tra cứu biểu mẫu (Template) tài liệu

- Giảng viên lưu ý sinh viên **không lên hỏi giảng viên về việc viết tài liệu như thế nào hay tìm template ở đâu**.
- Toàn bộ tài liệu tham khảo và biểu mẫu chuẩn đã được cung cấp đầy đủ trong mục tài liệu môn học trên hệ thống, sinh viên phải chủ động vào đó tải về để nghiên cứu và làm theo.

--- HẾT NỘI DUNG VĂN BẢN GHI ÂM (BẢO LƯU TOÀN VẸN 100% CHI TIẾT) ---